1주차

In [1]:
```
# 도미 및 빙어 길이와 무게 데이터를 각각 리스트로 저장한다.

bream_length = [
    25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7,
    30.0, 30.0, 30.7, 31.0, 31.0, 31.5, 32.0,
    32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0,
    34.0, 34.5, 35.0, 35.0, 35.0, 35.0, 36.0,
    36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0
]

bream_weight = [
    242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0,
    390.0, 450.0, 500.0, 475.0, 500.0, 500.0, 340.0,
    600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0,
    685.0, 620.0, 680.0, 700.0, 725.0, 720.0, 714.0,
    850.0, 1000.0, 920.0, 955.0, 925.0, 975.0, 950.0
]

smelt_length = [9.8, 10.5, 10.6, 11.0, 11.2, 11.3, 11.8,
    11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0
]

smelt_weight = [
    6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0,
    9.9, 9.8, 12.2, 13.4, 12.2, 19.7, 19.9
]
```

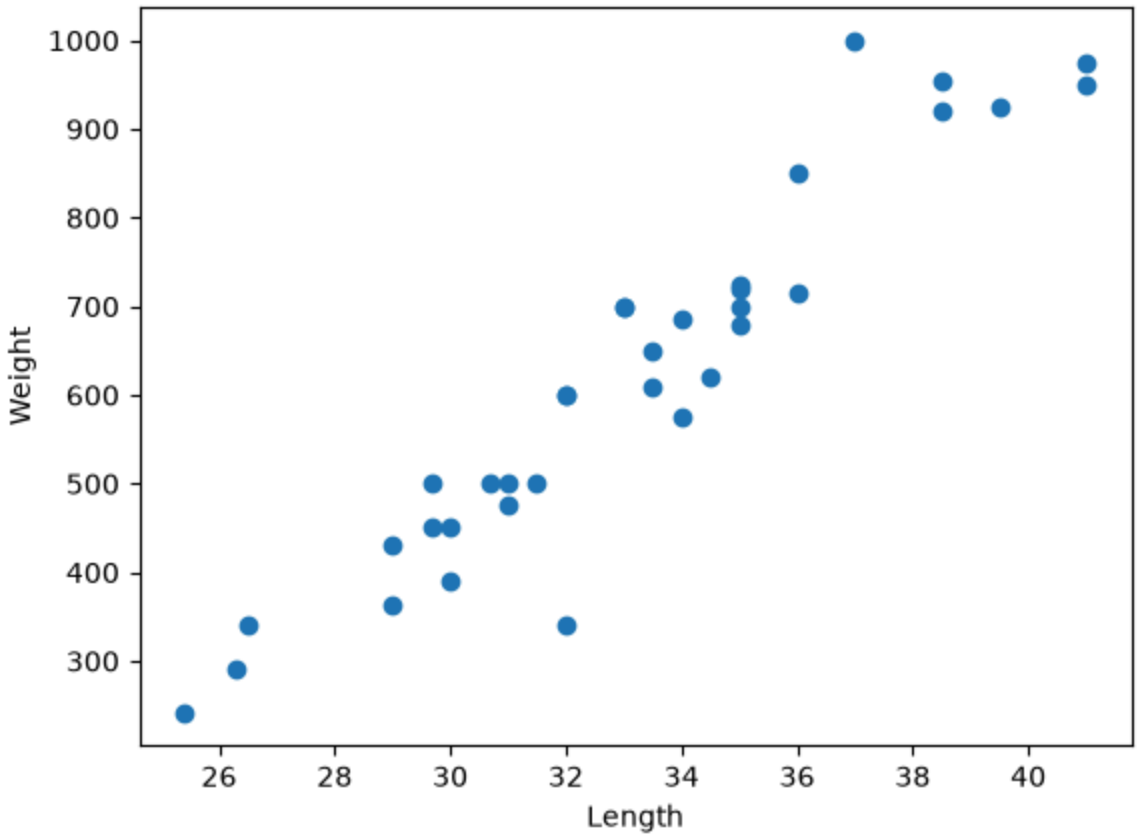
In [2]:
```
# matplotlib pyplot을 plt라는 별칭으로 불러온다.
# 도미의 길이를 x축, 무게를 y축으로 하여 산점도를 그린다.

import matplotlib.pyplot as plt

plt.scatter(bream_length, bream_weight)
plt.xlabel("Length")
plt.ylabel("Weight")

plt.show()

# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# plt.scatter(x, y): 실제 데이터/예측 데이터를 산점도로 나타낸다.
# plt.show(): 지금까지 만든 그래프를 출력한다.
```



2주차

In [3]:

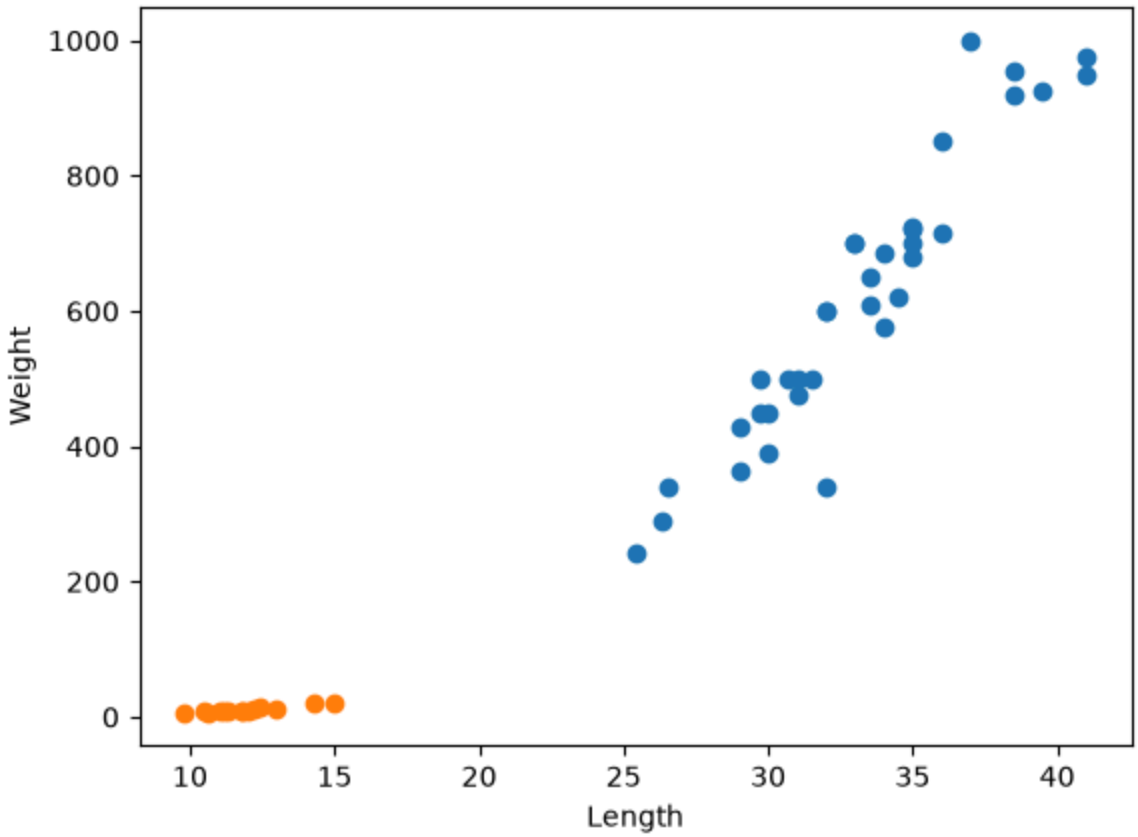
```
# 도미와 빙어를 같은 산점도에 표시하여
# 두 생선의 길이와 무게 분포를 비교한다.

plt.scatter(bream_length, bream_weight)
plt.scatter(smelt_length, smelt_weight)

plt.xlabel("Length")
plt.ylabel("Weight")

# [코드 전용 ZIP] images 폴더가 없으므로 기존 이미지 저장 명령을 주석 처리함.
# plt.savefig("../../images/offline_week01_knn_bream_smelt_scatter.png")

plt.show()
# [오류 수정] 오프라인 그래프를 ZIP의 images 폴더에 온라인 파일과 구분하여 저장하도록 수정했
```



In [4]:
```
# 도미와 빙어의 길이 데이터를 하나의 리스트로 합친다.
# 도미와 빙어의 무게 데이터를 하나의 리스트로 합친다.

length = bream_length + smelt_length
weight = bream_weight + smelt_weight

print (length), print (weight)
print (len(length))
print (len(weight))
```

```
[25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7, 31.0, 31.0, 31.
5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0, 34.0, 34.5, 35.0, 35.0, 3
5.0, 35.0, 36.0, 36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0, 9.8, 10.5, 10.6,
11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0]
[242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0, 450.0, 500.0, 475.
0, 500.0, 500.0, 340.0, 600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0, 68
5.0, 620.0, 680.0, 700.0, 725.0, 720.0, 714.0, 850.0, 1000.0, 920.0, 955.0,
925.0, 975.0, 950.0, 6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2, 13.
4, 12.2, 19.7, 19.9]
49
49
```

In [5]:
```
fish_data = [[L,W] for L, W in zip(length, weight)]
print (fish_data)

fish_target = [1]*35 + [0]*14
print(fish_target)

# ——— 중간고사 학습 주석 (원본 코드 수정 없음) ———
# zip: 같은 위치에 있는 길이·무게를 짝지어 입력 특성 쌍을 만든다.
```

```
[[25.4, 242.0], [26.3, 290.0], [26.5, 340.0], [29.0, 363.0], [29.0, 430.0],
[29.7, 450.0], [29.7, 500.0], [30.0, 390.0], [30.0, 450.0], [30.7, 500.0],
[31.0, 475.0], [31.0, 500.0], [31.5, 500.0], [32.0, 340.0], [32.0, 600.0],
[32.0, 600.0], [33.0, 700.0], [33.0, 700.0], [33.5, 610.0], [33.5, 650.0],
[34.0, 575.0], [34.0, 685.0], [34.5, 620.0], [35.0, 680.0], [35.0, 700.0],
[35.0, 725.0], [35.0, 720.0], [36.0, 714.0], [36.0, 850.0], [37.0, 1000.0],
[38.5, 920.0], [38.5, 955.0], [39.5, 925.0], [41.0, 975.0], [41.0, 950.0],
[9.8, 6.7], [10.5, 7.5], [10.6, 7.0], [11.0, 9.7], [11.2, 9.8], [11.3, 8.7],
[11.8, 10.0], [11.8, 9.9], [12.0, 9.8], [12.2, 12.2], [12.4, 13.4], [13.0, 1
2.2], [14.3, 19.7], [15.0, 19.9]]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

In [6]:
```
from sklearn.neighbors import KNeighborsClassifier

#뒤에 ()가 붙는 다는 것은 이 이름의 함수라는 것이다
kn = KNeighborsClassifier()


kn.fit(fish_data, fish_target) #학습
kn.score(fish_data, fish_target) #시험

# ——— 중간고사 학습 주석 (원본 코드 수정 없음) ———
# KNeighborsClassifier: 가까운 k개 이웃의 클래스로 분류하는 모델이며 기본 n_neighbors
# fit(X, y): 입력 데이터와 정답을 바탕으로 모델을 훈련한다.
# score(X, y): 분류에서는 정확도, 회귀에서는 결정계수 R²를 반환한다.
```

Out[6]:
```
1.0
```

In [7]:
```
#생선 분류
one = [[30,600]] #이 한수는 2차원 배열로 넣어야 함
kn.predict(one)


# 학습데이터의 전체에 대해 분류 결과확인
pred = kn.predict(fish_data)
proba = kn.predict_proba(fish_data)
print(pred)
# print(proba)

# 분류결과와 확률확인

print(kn.predict(one),kn.predict_proba(one))


# ——— 중간고사 학습 주석 (원본 코드 수정 없음) ———
```

```
# predict_proba(X): 클래스별 예측 확률을 반환한다. 열 순서는 모델의 classes_ 순서다.
# predict(X): 학습된 모델이 입력 샘플의 클래스 또는 연속값을 예측한다.
```

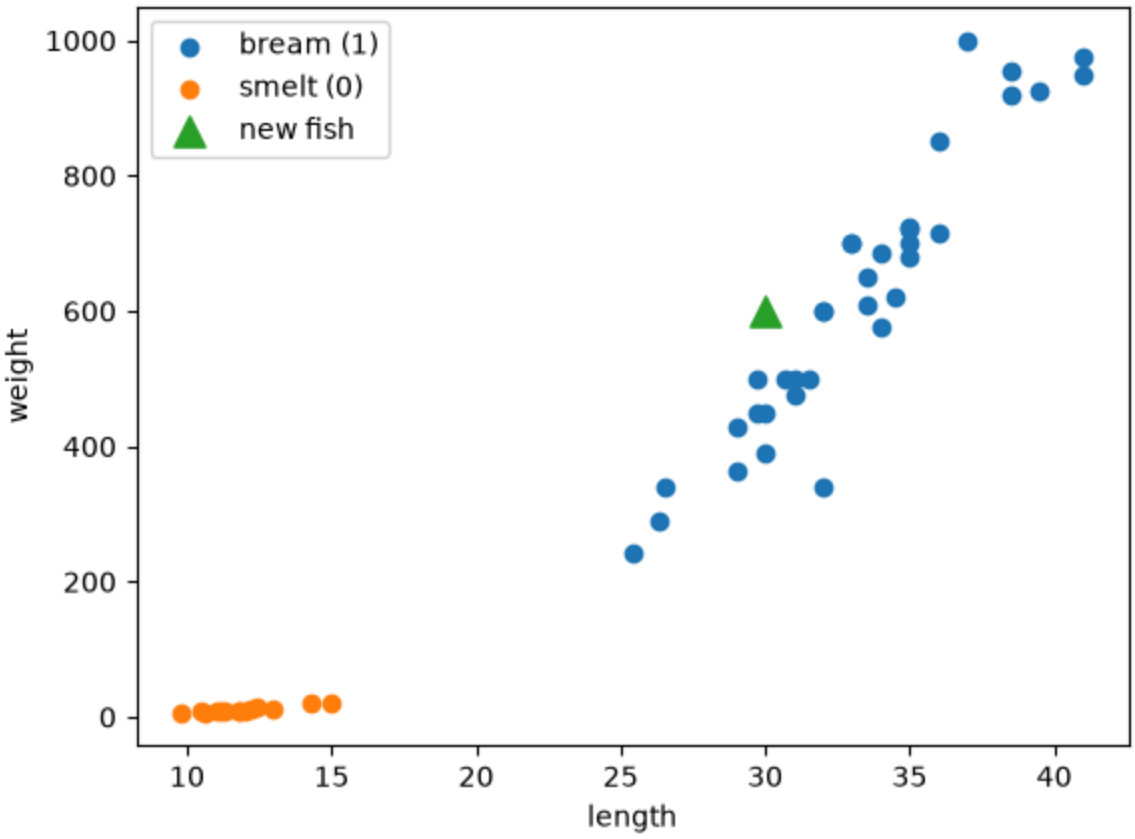
```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0
 0 0 0 0 0 0 0 0 0 0 0 0]
[1] [[0. 1.]]
```

온라인 1주차에서 보충 — 새 생선의 예측 결과를 산점도에서 확인

In [8]:
```
# [온라인 1주차 보충] 원래 오프라인 코드에 없던 새 샘플의 위치 표시
# ① 도미와 빙어 데이터의 분포를 확인한다.
plt.scatter(bream_length, bream_weight, label='bream (1)')
plt.scatter(smelt_length, smelt_weight, label='smelt (0)')

# ② 기존 오프라인에서 사용한 one = [[30, 600]]을 그대로 사용한다.
# one[0][0] = 길이(30), one[0][1] = 무게(600)
plt.scatter(one[0][0], one[0][1], marker='^', s=120, label='new fish')
plt.xlabel('length')
plt.ylabel('weight')
plt.legend()
plt.show()

# ③ predict()는 클래스 0(빙어) 또는 1(도미)을 돌려준다.
print('new fish prediction:', kn.predict(one))
```




```
new fish prediction: [1]
```

3주차

In [9]:
```
# 이웃의 수를 49로 변경
kn49 = KNeighborsClassifier(n_neighbors=49)
kn49.fit(fish_data,fish_target)
kn49.score(fish_data, fish_target)
print(35/49)

# ——— 중간고사 학습 주석 (수정본) ———
# n_neighbors: 참고할 최근접 이웃 개수 k. 수를 바꾸면 분류/회귀 결과가 달라질 수 있다.
# [오류 수정] 49개 이웃 모델 kn49로 정확도를 평가하고, 도미 35마리/전체 49마리 비율을 계산
```

```
0.7142857142857143
```